Everything Claude Code (ECC): Eine technische Einführung für Einsteiger

Eine Hausarbeit über das agentenbasierte KI-Optimierungssystem

Datum: März 2026
Autor: KI-Assistent für Technologie-Einsteiger
Thema: Architektur und Funktionsweise des ECC-Repository
Zielgruppe: Tech-Anfänger mit Grundverständnis von Softwareentwicklung

---

Inhaltsverzeichnis

1. Einleitung: Was ist ECC?
2. Das Problem: Warum brauchen wir so ein System?
3. Die fünf Säulen der ECC-Architektur
    - 3.1 Agents (Spezialisierte Unterassistenten)
    - 3.2 Skills (Wissenspakete)
    - 3.3 Commands (Schnelle Befehle)
    - 3.4 Hooks (Automatisierung)
    - 3.5 Rules (Regelwerke)
4. Wie alles zusammenspielt: Der ECC-Workflow
5. Besondere Funktionen im Detail
    - 5.1 Continuous Learning v2
    - 5.2 Selective Install
    - 5.3 Token-Optimierung
    - 5.4 Cross-Platform-Support
6. Sicherheitsaspekte
7. Fazit und Ausblick

1. Einleitung: Was ist ECC?

Everything Claude Code (ECC) ist ein umfassendes Optimierungssystem für KI-gestützte Softwareentwicklung. Stellen Sie es sich als ein **erweitertes Werkzeugkasten-System** für Claude Code (einen KI-Assistenten von Anthropic) vor – vergleichbar mit einem Profi-Werkzeugkasten, der einem Handwerker nicht nur mehr Werkzeuge gibt, sondern auch beibringt, wann welches Werkzeug zum Einsatz kommt.

Das Repository enthält:

- **28 spezialisierte Agenten** (Unterassistenten)
- **116 Skills** (Domänenwissen)
- **59 Commands** (Slash-Befehle)
- **178 Skripte** (Hooks und Utilities)
- **34 Regelwerke** für verschiedene Programmiersprachen

ECC wurde von Affaan Mustafa bei einem Anthropic-Hackathon entwickelt und hat mittlerweile über 50.000 Sterne auf GitHub. Es ist kein einfaches Konfigurationspaket, sondern ein komplettes **Agenten-Harness-Performance-System**.

Was bedeutet "Agenten-Harness"?

Ein "Harness" (Gurtzeug) ist im Englischen das Sicherheitsgeschirr, das Kletterer tragen. Übertragen auf die Software bedeutet es: Ein System, das den KI-Agenten (Claude) **führt, unterstützt und sicher hält**, während er komplexe Aufgaben erledigt.

---

2. Das Problem: Warum brauchen wir so ein System?

Um ECC zu verstehen, müssen wir zuerst das Problem verstehen, das es löst.

2.1 Das Kontext-Limit

Moderne KI-Systeme wie Claude haben ein **Kontextfenster** – eine Begrenzung, wie viel Information sie gleichzeitig verarbeiten können (bei Claude ca. 200.000 Tokens, was ungefähr 150.000 Wörtern entspricht).

Das Problem: Bei langen Coding-Sessions füllt sich dieses Fenster mit:

- Gelesenen Dateien
- Tool-Ausgaben
- Fehlermeldungen
- Konversationsverlauf

Wenn das Fenster voll ist, beginnt die KI, ältere Informationen zu "vergessen" – mit schlechteren Ergebnissen.

2.2 Token-Kosten

Jede Interaktion mit einer KI kostet **Tokens** (Recheneinheiten). Je komplexer das Modell (z.B. Claude Opus vs. Haiku), desto teurer. Ohne Optimierung können schnell Tausende von Dollar an API-Kosten entstehen.

2.3 Wiederholende Fehler

Programmierer machen oft dieselben Fehler:

- Vergessene Tests
- Sicherheitslücken
- Inkonsistenter Code-Stil
- `console.log` Statements, die im Produktivcode landen

2.4 Kontextverlust zwischen Sessions

Wenn Sie Claude schließen und später wieder öffnen, hat er den vorherigen Kontext vergessen. Alle Erkenntnisse, Architekturentscheidungen und Projekt-spezifischen Konventionen müssen neu erklärt werden.

2.5 Die Lösung: ECC

ECC adressiert all diese Probleme durch:

Problem	ECC-Lösung
Kontext-Limit	Strategische Kompaktierung, Subagenten

Problem	ECC-Lösung
Token-Kosten	Modell-Routing (Haiku → Sonnet → Opus)
Wiederholende Fehler	Hooks für automatische Qualitätsprüfung
Kontextverlust	Memory-Persistence über Sessions
Wissenslücken	116 Skills für verschiedene Domänen

---

3. Die fünf Säulen der ECC-Architektur

ECC baut auf fünf miteinander verbundenen Komponenten auf. Stellen Sie sich diese wie die Säulen eines Tempels vor – jede hat ihre eigene Funktion, aber zusammen tragen sie das gesamte Gewicht.

3.1 Agents (Spezialisierte Unterassistenten)

Was sind Agents?

Agents sind **spezialisierte Unterassistenten**, die für bestimmte Aufgaben optimiert sind. Statt einen "Generalisten" für alles zu verwenden, delegiert Claude an Spezialisten – ähnlich wie in einem Unternehmen, wo der CEO an Fachabteilungen delegiert.

Die 28 ECC-Agenten:

Agent	Spezialisierung	Wann eingesetzt?
`planner`	Implementierungsplanung	Neue Features, Architekturänderungen
`code-reviewer`	Code-Qualitätsprüfung	Nach Implementierung
`security-reviewer`	Sicherheitsaudit	Vor Produktivsetzung

Agent	Spezialisierung	Wann eingesetzt?
`tdd-guide`	Test-Driven Development	Beim Schreiben neuer Features
`build-error-resolver`	Build-Fehler beheben	Bei Kompilierungsfehlern
`e2e-runner`	End-to-End-Tests	Playwright-Testautomatisierung
`typescript-reviewer`	TypeScript-Code-Review	TS/JS-Projekte
`python-reviewer`	Python-Code-Review	Python-Projekte
`go-reviewer`	Go-Code-Review	Go-Projekte
`refactor-cleaner`	Tote Code-Bereinigung	Refactoring

Wie funktioniert ein Agent?

Ein Agent ist eine Markdown-Datei mit spezieller Struktur:

```
---
name: planner
description: Expert planning specialist for complex features
tools: ["Read", "Grep", "Glob"]
model: opus
---
```

Diese YAML-Frontmatter definiert:

- **Name:** Wie der Agent heißt
- **Description:** Wann Claude diesen Agenten einsetzen soll
- **Tools:** Welche Werkzeuge der Agent verwenden darf
- **Model:** Welches KI-Modell verwendet wird (Opus für komplexe Planung)

Der Rest der Datei enthält **Instruktionen**, die dem Agenten genau sagen, wie er vorgehen soll – vergleichbar mit einer detaillierten Arbeitsanweisung.

Warum verschiedene Modelle?

Modell	Kosten	Beste für
Haiku	Niedrig	Einfache Aufgaben, Datei-Lesen
Sonnet	Mittel	Tägliche Programmierung
Opus	Hoch	Komplexe Architektur, Debugging

ECC wählt automatisch das passende Modell – das spart bis zu 80% der Kosten.

3.2 Skills (Wissenspakete)

Was sind Skills?

Skills sind **Domänenwissen in Paketen** – vergleichbar mit Fachbüchern, die Claude bei Bedarf zurate ziehen kann. Ein Skill ist eine Markdown-Datei, die einen bestimmten Workflow oder ein technisches Gebiet dokumentiert.

Beispiele für Skills:

Skill	Inhalt
`tdd-workflow`	Test-Driven Development: RED-GREEN-REFACTOR Zyklus
`django-patterns`	Best Practices für Django-Framework
`security-review`	Sicherheits-Checkliste (OWASP Top 10)
`frontend-patterns`	React/Next.js Architekturmuster
`postgres-patterns`	PostgreSQL-Optimierungen
`continuous-learning-v2`	Automatisches Lernen aus Sessions

Struktur eines Skills:

```
 ---
name: tdd-workflow
description: Test-driven development methodology
---

# TDD Workflow

1. Define interfaces first
2. Write failing tests (RED)
3. Implement minimal code (GREEN)
4. Refactor (IMPROVE)
5. Verify 80%+ coverage
```

Der Unterschied zu Agents:

Agents	Skills
"Wer" führt die Aufgabe aus	"Wie" wird die Aufgabe erledigt
Aktive Ausführung	Passives Wissen
Haben Tools und Modell	Sind reine Dokumentation
Werden gestartet	Werden referenziert

3.3 Commands (Schnelle Befehle)

Was sind Commands?

Commands sind **Slash-Befehle** (wie `/plan`, `/tdd`, `/code-review`), die der Benutzer direkt eingeben kann, um sofort eine Aktion auszulösen. Sie sind die Benutzerschnittstelle zu Agents und Skills.

Wichtige Commands:

Command	Funktion	Agent/Skill
`/plan "Feature"`	Erstellt Implementierungsplan	planner

Command	Funktion	Agent/Skill
`/tdd`	Startet TDD-Workflow	tdd-guide
`/code-review`	Überprüft Code	code-reviewer
`/build-fix`	Behebt Build-Fehler	build-error-resolver
`/security-scan`	Sicherheitsaudit	security-reviewer
`/e2e`	E2E-Tests generieren	e2e-runner
`/learn`	Muster aus Session extrahieren	continuous-learning
`/compact`	Kontext komprimieren	strategic-compact

Wie Commands funktionieren:

Wenn Sie `/plan "Add authentication"` eingeben:

1. Claude erkennt den Command
2. Lädt die Definition aus `commands/plan.md`
3. Delegiert an den `planner`-Agenten
4. Der Agent erstellt den Plan
5. Ergebnis wird angezeigt

3.4 Hooks (Automatisierung)

Was sind Hooks?

Hooks sind **automatisch ausgeführte Skripte**, die bei bestimmten Ereignissen (Events) feuern. Sie sind wie automatische Qualitätskontrollen, die im Hintergrund laufen.

Die fünf Hook-Typen:

Hook-Typ	Wann feuert?	Kann blockieren?
PreToolUse	Vor Tool-Ausführung	Ja
PostToolUse	Nach Tool-Ausführung	Nein
Stop	Nach Claude-Antwort	Nein
SessionStart	Bei Session-Beginn	Nein
PreCompact	Vor Kontext-Kompaktierung	Nein

Beispiele für Hooks:

Hook	Event	Was tut es?
Dev Server Blocker	PreToolUse	Blockiert `npm run dev` außerhalb von tmux
Git Push Reminder	PreToolUse	Erinnert an Review vor `git push`
console.log Warning	PostToolUse	Warnt bei `console.log` im Code
TypeScript Check	PostToolUse	Führt `tsc --noEmit` nach TS-Änderungen
Prettier Format	PostToolUse	Formatiert Code automatisch
Session Start	SessionStart	Lädt vorherigen Kontext

Der Dev Server Blocker erklärt:

```
 // Hook-Logik vereinfacht:
if (tool === "Bash" && command.includes("npm run dev")) {
  if (!isInsideTmux()) {
    console.error("[Hook] Dev-Server außerhalb von tmux blockiert");
    process.exit(2); // Blockiert!
  }
}
```

Dieser Hook stellt sicher, dass langlebige Prozesse (wie Entwicklungsserver) in einer tmux-Session laufen, damit sie nicht verloren gehen, wenn die Verbindung unterbrochen wird.

Hook-Profile:

Profil	Bedeutung
`minimal`	Nur essentielle Hooks
`standard`	Balancierte Qualität + Sicherheit (Default)
`strict`	Strengere Prüfungen

Steuerbar via: `export ECC_HOOK_PROFILE=minimal`

3.5 Rules (Regelwerke)

Was sind Rules?

Rules sind **immer gültige Richtlinien**, die Claude bei jeder Interaktion befolgen soll. Sie sind vergleichbar mit Firmenrichtlinien oder Styleguides.

Struktur der Rules:

```
rules/
├── common/           # Sprach-unabhängige Regeln
│   ├── coding-style.md
│   ├── testing.md
│   ├── git-workflow.md
│   ├── security.md
│   └── performance.md
├── typescript/       # TypeScript-spezifisch
├── python/           # Python-spezifisch
├── golang/           # Go-spezifisch
└── php/              # PHP-spezifisch
```

Beispiel-Regel (testing.md):

```
# Testing Requirements

- Always write tests before implementation (TDD)
- Maintain 80%+ code coverage
- Test edge cases: null, empty, maximum values
- Use descriptive test names: `should_throw_when_invalid_input`
- Mock external dependencies
```

Wichtige Eigenschaft: Rules werden **nicht** über das Plugin-System verteilt (Claude-Limitation). Sie müssen manuell installiert werden.

---

4. Wie alles zusammenspielt: Der ECC-Workflow

Jetzt verbinden wir alle fünf Säulen zu einem Gesamtbild. Stellen Sie sich ECC als **Orchester** vor, bei dem jedes Instrument (Komponente) seine Rolle spielt.

Der typische ECC-Entwicklungszyklus:

```
┌─────────────────────────────────────────────────────────────────────┐
│                      ECC WORKFLOW DIAGRAM                           │
└─────────────────────────────────────────────────────────────────────┘


1. SESSION START
   │
   ├── SessionStart-Hook lädt vorherigen Kontext
   ├── Package Manager wird erkannt
   └── Rules werden geladen
   │
   ▼
2. FEATURE-ANFRAGE
   │
   User: "Füge Benutzer-Authentifizierung hinzu"
   │
   ▼
3. PLANUNG (/plan)
   │
   ├── Command erkannt: /plan
   ├── planner-Agent wird gestartet (Opus-Modell)
   ├── planner nutzt tdd-workflow Skill
   └── Ergebnis: Detaillierter Implementierungsplan
```

```
    │
    ▼
4. IMPLEMENTIERUNG (/tdd)
    │
    ├── Command: /tdd
    ├── tdd-guide Agent (Sonnet)
    ├── RED: Test schreiben → Test fehlschlägt
    ├── GREEN: Code implementieren → Test besteht
    └── IMPROVE: Refactoring
    │
    ├── PreToolUse Hooks prüfen:
    │   ├── Dev Server Blocker (wenn npm run dev)
    │   └── Git Push Reminder (wenn git push)
    │
    ├── PostToolUse Hooks:
    │   ├── console.log Warning
    │   ├── TypeScript Check
    │   └── Prettier Format
    │
    ▼
5. CODE REVIEW (/code-review)
    │
    ├── code-reviewer Agent
    ├── Nutzt security-review Skill
    └── Prüft: Qualität, Sicherheit, Maintainability
    │
    ▼
6. KONTEXT-VERWALTUNG
    │
    ├── Strategische Kompaktierung (/compact)
    ├── PreCompact-Hook speichert Zustand
    └── Observer extrahiert Patterns für Continuous Learning
    │
    ▼
7. SESSION ENDE
    │
    ├── SessionEnd-Hook speichert Zustand
    └── Continuous Learning: Instincts werden erstellt
```

Die Zusammenhänge im Detail:

1. Rules → Alle Komponenten Rules sind das Fundament. Sie gelten für alle Agents, Commands und Hooks.

2. Skills → Agents Agents nutzen Skills als Wissensquelle. Der `tdd-guide` Agent verwendet den `tdd-workflow` Skill.

3. Commands → Agents Commands sind der Einstiegspunkt. `/tdd` startet den `tdd-guide` Agenten.

4. Hooks → Automatisierung Hooks überwachen und steuern den gesamten Prozess im Hintergrund.

5. Continuous Learning → Alles Das System lernt aus jeder Session und verbessert sich selbst.

---

5. Besondere Funktionen im Detail

5.1 Continuous Learning v2

Das Konzept:

ECC beobachtet jede Session und lernt daraus. Es erstellt **"Instincts"** (Instinkte) – atomare Verhaltensweisen mit Konfidenz-Scoring.

Beispiel:

Wenn Sie in mehreren Sessions Code-Reviews ablehnen, die `console.log` enthalten, lernt ECC:

```
 ---
id: avoid-console-log
trigger: "when editing TypeScript files"
confidence: 0.85
domain: "code-quality"
---

# Avoid console.log

## Action
Warn about console.log statements in production code.

## Evidence
- User rejected 5 PRs with console.log (2026-01-15 bis 2026-01-20)
- User explicitly said "remove all console.logs"
```

V2.1 Neuerung – Project-Scoped Instincts:

- React-Patterns bleiben im React-Projekt
- Python-Konventionen bleiben im Python-Projekt
- Universelle Patterns (z.B. "validiere Input") werden global geteilt

Der Lernprozess:

```
 Session-Aktivität
      │
      ▼
Hooks erfassen Tool-Usage (100% zuverlässig)
      │
      ▼
observations.jsonl (gespeichert)
      │
      ▼
Observer-Agent (Haiku, im Hintergrund)
      │
      ▼
Pattern-Erkennung:
├── Benutzer-Korrekturen → Instinct
├── Fehler-Lösungen → Instinct
└── Wiederholte Workflows → Instinct
      │
      ▼
Instincts.json (mit Confidence-Scoring)
      │
      ▼
/evolve → Cluster zu Skills/Commands/Agents
```

5.2 Selective Install

Das Problem:

Nicht jeder braucht alles. Ein Python-Entwickler benötigt keine TypeScript-Rules.

Die Lösung:

Manifest-basierte Installation:

```
# Nur TypeScript installieren
./install.sh typescript

# Mehrere Sprachen
./install.sh typescript python golang

# Für Cursor IDE
./install.sh --target cursor typescript
```

Die Architektur:

```
manifests/
├── install-modules.json      # Verfügbare Module
└── install-profiles.json     # Vordefinierte Profile

schemas/
├── install-modules.schema.json
├── install-profiles.schema.json
└── install-state.schema.json
```

Install-Pipeline:

1. **CLI** → Benutzer-Input
2. **Request Normalization** → Kanonische Anfrage
3. **Module Resolution** → Abhängigkeiten auflösen
4. **Target Planning** → Ziel-spezifische Planung
5. **Operation Planning** → Was soll passieren?
6. **Execution** → Dateien kopieren/generieren
7. **State Persistence** → Was wurde installiert?

Lifecycle-Commands:

Command	Funktion
`ecc plan`	Zeigt Installationsplan vorher
`ecc doctor`	Diagnostiziert Installation
`ecc repair`	Stellt fehlende Dateien wieder her

Command	Funktion
`ecc uninstall`	Entfernt ECC-Dateien
`ecc status`	Zeigt Installations-Status

5.3 Token-Optimierung

Strategien:

Strategie	Einsparung
Modell-Routing (Haiku → Sonnet → Opus)	~60%
MAX_THINKING_TOKENS=10000	~70%
Autocompact bei 50%	Bessere Session-Qualität
Subagent-Modell: Haiku	~80%
MCP-Limit (max 10 Server)	Verhindert Kontext-Explosion

Empfohlene Settings:

```
{
  "model": "sonnet",
  "env": {
    "MAX_THINKING_TOKENS": "10000",
    "CLAUDE_AUTOCOMPACT_PCT_OVERRIDE": "50",
    "CLAUDE_CODE_SUBAGENT_MODEL": "haiku"
  }
}
```

5.4 Cross-Platform-Support

ECC funktioniert über mehrere Plattformen:

Plattform	Unterstützung	Besonderheiten
Claude Code	Nativ	Primäres Ziel
Cursor IDE	Voll	15+ Hook-Events
OpenCode	Voll	20+ Plugin-Events
Codex CLI/App	Voll	AGENTS.md-basiert
Antigravity	Voll	`.agent/`-Integration

Die Adapter-Pattern:

```
Cursor stdin JSON → adapter.js → transformiert →
ECC-Standardformat → scripts/hooks/*.js
```

Dies ermöglicht Code-Wiederverwendung über Plattformen hinweg.

---

6. Sicherheitsaspekte

6.1 Risiko-Kategorien

Risiko	Beschreibung	Mitigation
Datensammlung	`observe.sh` sammelt Tool-Usage	Profil-gated, deaktivierbar
Code-Ausführung	Hooks können beliebigen Code ausführen	Path-Traversal-Checks
Secrets-Exposure	API-Keys in Logs	Auto-Redaktion in observe.sh

Risiko	Beschreibung	Mitigation
Supply Chain	Install-Skripte	Manuelle Überprüfung möglich

6.2 Schutzmechanismen

Environment Variables:

```
# Deaktiviert nicht-essentielle Hooks
export ECC_HOOK_PROFILE=minimal

# Überspringt Beobachtung
export ECC_SKIP_OBSERVE=1

# Deaktiviert spezifische Hooks
export ECC_DISABLED_HOOKS="pre:bash:tmux-reminder"
```

5-Layer Guard im observe.sh:

1. Entrypoint-Check (nur CLI/SDK)
2. Hook-Profile (minimal deaktiviert)
3. Env-Var (ECC_SKIP_OBSERVE)
4. Agent-ID (keine Subagenten)
5. Path-Exclusions (observer-sessions)

6.3 AgentShield

ECC enthält ein Sicherheits-Audit-Tool:

```
# Schneller Scan
npx ecc-agentshield scan

# Mit Auto-Fix
npx ecc-agentshield scan --fix

# Deep Analysis (3 Opus Agenten)
npx ecc-agentshield scan --opus
```

Prüft: Secrets, Permissions, Hook-Injection, MCP-Risiken, Agent-Configs.

7. Fazit und Ausblick

Was haben wir gelernt?

ECC ist kein einfaches Plugin-Paket, sondern ein **komplexes System** zur Optimierung von KI-gestützter Softwareentwicklung. Die fünf Säulen (Agents, Skills, Commands, Hooks, Rules) bilden ein zusammenhängendes Ganzes:

- **Agents** bringen Spezialisierung
- **Skills** bringen Domänenwissen
- **Commands** bringen Benutzerfreundlichkeit
- **Hooks** bringen Automatisierung
- **Rules** bringen Konsistenz

Die Kerninnovationen:

1. **Continuous Learning v2** – Das System lernt aus jeder Session
2. **Selective Install** – Nur installieren, was man braucht
3. **Token-Optimierung** – Intelligentes Modell-Routing spart Kosten
4. **Cross-Platform** – Funktioniert über verschiedene IDEs
5. **Project-Scoped** – Isoliertes Lernen verhindert Kontamination

Ausblick auf ECC 2.0:

Die nächste Generation plant:

- **Rust-basierte TUI** (Terminal User Interface)
- **Native SSH-Unterstützung**
- **Workspace Runtime Registry**
- **Persistent Daemon**
- **Agent Teams** (parallele Multi-Agent-Workflows)

Für wen ist ECC geeignet?

Profil	Empfehlung
Einzelentwickler	Ja, sofort einsetzbar
Teams	Ja, durch Continuous Learning
Enterprise	Ja, mit AgentShield Security
Anfänger	Mit Vorsicht – komplexe Architektur

Abschließender Vergleich:

Aspekt	Ohne ECC	Mit ECC
Kontext-Management	Manuell	Automatisiert
Code-Qualität	Variabel	Konsistent
Wiederverwendung	Gering	Hoch (Skills)
Kosten	Höher	Optimierter
Lernen	Vergessen	Persistent

ECC repräsentiert den **State-of-the-Art** für KI-gestützte Entwicklungssysteme und zeigt, wie mächtig die Kombination aus guter Architektur und KI sein kann.

Glossar

Begriff	Erklärung
Agent	Spezialisierter Unterassistent für bestimmte Aufgaben
Command	Slash-Befehl zur direkten Ausführung

Begriff	Erklärung
Context Window	Speicherbereich der KI für aktuelle Konversation
Hook	Automatisch ausgeführtes Skript bei Ereignissen
Instinct	Gelernte atomare Verhaltensweise
MCP	Model Context Protocol (Schnittstelle zu externen Tools)
Skill	Paketiertes Domänenwissen
TUI	Terminal User Interface
Token	Recheneinheit für KI-Interaktionen

Ende der Hausarbeit

Erstellt im März 2026 als technische Einführung in das Everything Claude Code Repository.